COURSE: CSC 221
DATA STRUCTURES AND ALGORITHM
PROJECT REPORT

CLASS: BSE – 3C (FALL - 2023)

Random Maze Generator

Group Members

Student Name	Enrollment#
Taiba Hussain **(Team Lead)**	02-131222-078
Eeshal Hassan	02-131222-100

Submitted to:

Course Instructor: Engr. Majid Kalim
Lab Instructor: Engr. Ayesha Khan

Department of Software Engineering
BAHRIA UNIVERSITY KARACHI CAMPUS

TABLE OF CONTENTS

1. Introduction **3**

2. Problem **3**

3. Paradigms **3**

4. Algorithm And Explanation **3**

4.1. Randomized Kruskal's Algorithm (Maze Generation) 3

4.2. A* Algorithm (Maze Solving) 3

5. Algorithm Code **4**

6. Interfaces **7**

7. Conclusion **8**

1. Introduction

The "Random Maze Generator" project, named Maze Runner, is a console-based tool designed for generating, solving, and visualizing mazes. This project leverages randomized Kruskal's algorithm for maze generation and the A* algorithm for finding the optimal path from the entrance to the exit of the maze. Additionally, the tool provides functionality to save and load mazes in text format, enhancing user flexibility. Implemented in pure Java, the project maintains a comprehensive Javadoc for ease of understanding and future development.

2. Problem

The primary problem addressed by Maze Runner is the generation of random mazes and finding a solution path within them. Mazes are complex structures with intricate patterns, and Maze Runner aims to offer a versatile solution for maze creation and navigation. The project targets users who seek a tool to generate mazes of varying sizes and complexities and find optimal paths through them.

3. Paradigms

The project adheres to the object-oriented paradigm, utilizing Java as the programming language. Object-oriented principles such as encapsulation, inheritance, and polymorphism are employed to create a modular and maintainable codebase. The project also aligns with the paradigm of algorithmic problem-solving, implementing randomized Kruskal's algorithm and the A* algorithm to address maze generation and pathfinding challenges.

4. Algorithm And Explanation

4.1. Randomized Kruskal's Algorithm (Maze Generation)

The project employs the randomized Kruskal's algorithm for maze generation. This algorithm involves creating a spanning tree of the maze, where each cell is a node in the tree. The edges represent passages, and by randomly connecting nodes while avoiding cycles, a maze with a unique solution path is generated.

4.2. A* Algorithm (Maze Solving)

To find the optimal path from the entrance to the exit of the maze, the A* algorithm is utilized. This algorithm is a popular choice for pathfinding in graph-based scenarios. It uses a heuristic to

estimate the cost of reaching the destination from a specific point, efficiently guiding the search towards the optimal solution.

5. Algorithm Code

```
package maze.algo.generation;

import maze.model.Cell;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

import static java.util.stream.Collectors.toList;
import static maze.model.Cell.Type.PASSAGE;

/**
 * This class is used for creating random passages between
 * isolated passage cells such that every cell is connected
 * to the other in one way and the maze has no cycles.
 * Actually, it is a tree.
 * <p>
 * For example:
 * <pre>
 * ██  ██████
 * ██  ██  ██
 * ██████████
 * ██  ██  ██
 * ████████  ██
 * </pre>
 * can become
 * <pre>
 * ██  ██████
 * ██      ██
 * ████████  ██
 * ██      ██
 * ████████  ██
 * </pre>
 * <p>
 * Internally, it it creates a tree of edges between cells in a square
 * of doubly decreased size such that all cells are in the one
 * <a href="https://en.wikipedia.org/wiki/Component_(graph_theory)">
 * connected component</a>.
 * <p>
 * <a href="https://en.wikipedia.org/wiki/Maze_generation_algorithm#Randomized_Kruskal's_algorithm">
```

```
    /**
     * Creates a list of all possible edges in an imaginary edge form.
     *
     * @return a list of all possible edges
     * @see Edge
     */
    private List<Edge> createEdges() {
        var edges = new ArrayList<Edge>();
        for (int column = 1; column < width; column++) {
            edges.add(new Edge(toIndex(0, column),
                               toIndex(0, column - 1)));
        }
        for (int row = 1; row < height; row++) {
            edges.add(new Edge(toIndex(row, 0),
                               toIndex(row - 1, 0)));
        }
        for (int row = 1; row < height; row++) {
            for (int column = 1; column < width; column++) {
                edges.add(new Edge(toIndex(row, column),
                                   toIndex(row, column - 1)));
                edges.add(new Edge(toIndex(row, column),
                                   toIndex(row - 1, column)));
            }
        }
        return edges;
    }

    /**
     * Transforms the coordinates in a 2-dimensional array
     * to the coordinate in a 1-dimensional array using the
     * {@code row * width + column} formula.
     *
     * @param row    the row coordinate in a 2-dimensional array
     * @param column the column coordinate in a 2-dimensional array
     * @return the coordinate in a 1-dimensional array
     */
    private int toIndex(int row, int column) {
        return row * width + column;
    }

    /**
     * Generates a list of edges that connect passages. It is a
```

```
 * Randomized Kruskal's algorithm</a> is used.
 * <p>
 * Initially, there is a set of all possible edges in the small grid.
 * On each step of the algorithm it removes one of the edges such
 * that an edge connects two distinct connected components.
 * At the end, each cell is connected to the others in a one way.
 * <p>
 * For example, ({@code width = 9} and {@code height = 9}) were given.
 * It corresponds with the following maze where the entrance, the exit
 * and an every second cell are passages while others are walls:
 * <pre>
 * ██ ▄▄▄▄▄▄▄▄▄▄▄▄▄▄
 * ▀▀ ▀▀ ▀▀ ▀▀ ▀▀
 * ▄▄▄▄▄▄▄▄▄▄▄▄▄▄▄▄▄▄
 * ▀▀ ▀▀ ▀▀ ▀▀ ▀▀
 * ▄▄▄▄▄▄▄▄▄▄▄▄▄▄▄▄▄▄
 * ▀▀ ▀▀ ▀▀ ▀▀ ▀▀
 * ▄▄▄▄▄▄▄▄▄▄▄▄▄▄▄▄▄▄
 * ▀▀ ▀▀ ▀▀ ▀▀ ▀▀
 * ▄▄▄▄▄▄▄▄▄▄▄▄▄▄ ▄▄
 * </pre>
 * It is needed to connect these passages to make a simply connected
 * maze. After the size is decreased grid turns into the following
 * imaginary form in terms of edges:
 * <pre>
 *  _____
 * |_|_|_|_|
 * |_|_|_|_|
 * |_|_|_|_|
 * |_|_|_| |
 * </pre>
 * And, after "deleting" some random edges, a maze becomes:
 * <pre>
 *  _____
 * | |___  |
 * | |  _  |
 * |__ |   |
 * |___|_| |
 * </pre>
 * <p>
 * It then is scaled back to:
 * <pre>
 * ██ ▄▄▄▄▄▄▄▄▄▄▄▄▄▄
 * ▀▀ ▀▀       ▀▀
```

```
 *  <a href="https://en.wikipedia.org/wiki/Maze_generation_algorithm#Randomized_Kruskal's_algorithm">
 * Randomized Kruskal's algorithm</a> implementation.
 * <p>
 * On each step of the algorithm an edge is added to the list
 * only if it connects two disjoint subsets.
 *
 * @param edges a randomized list of all edges
 * @return a random list of edges that connect passages
 * @see DisjointSet
 */
private List<Edge> buildRandomSpanningTree(List<Edge> edges) {
    var disjointSets = new DisjointSet(width * height);
    return edges
        .stream()
        .filter(edge -> connects(edge, disjointSets))
        .collect(toList());
}

/**
 * Checks if an {@code edge} connects two disjoint subsets.
 *
 * @param edge        a given edge
 * @param disjointSet a disjoint-set data structure that keeps track of subsets
 * @return true if an edge connects two disjoint subsets,
 * false otherwise
 */
private boolean connects(Edge edge, DisjointSet disjointSet) {
    return disjointSet.union(edge.getFirstCell(), edge.getSecondCell());
}

/**
 * Scales and converts edges in an imaginary edge form to the cells
 * which connect passages in a original form.
 *
 * @param spanningTree a random list of edges that connect passages
 * @return a list of cells that connect passages
 */
private List<Cell> createPassages(List<Edge> spanningTree) {
    return spanningTree
        .stream()
```

```
 * ██  └──────┘  └──┘
 * └──┘ ┌──┐     ┌──┐
 * └──┘ └──┘ └──┘└──┘ └──┘
 * └──┘    └──┘    └──┘
 * └──┘└──┘ ┌──┐ ┌──┐ ┌──┐
 * └──┘    └──┘ └──┘ └──┘
 * └──────────────┘  └──┘
 * </pre>
 * <p>
 * The algorithm output is a list of cells that connect passages
 * in an original form.
 *
 * @author Philipp Malkovsky
 */

public class PassageTree
{
    /**
     * The height of the maze in an imaginary edge form.
     */
    private int height;

    /**
     * The width of the maze in an imaginary edge form.
     */
    private int width;

    /**
     * Creates a new imaginary edge form
     *
     * @param height an original height
     * @param width  an original width
     */
    public PassageTree(int height, int width) {
        this.height = (height - 1) / 2;
        this.width = (width - 1) / 2;
    }
    /**
     * Generates a random list of cells that connect passages in
     * an original form such that a maze is simply connected.
     *
     * @return a random list of cells that connect passages
     */
    public List<Cell> generate() {
        var edges = createEdges();
        Collections.shuffle(edges);
        var tree = buildRandomSpanningTree(edges);
        return createPassages(tree);
    }
```

```
        .map(edge -> {
            var first = fromIndex(edge.getFirstCell());
            var second = fromIndex(edge.getSecondCell());
            return getPassage(first, second);
        }).collect(toList());
    }

    /**
     * Transforms the coordinate in a 1-dimensional array
     * back to the coordinates in a 2-dimensional array using the
     * {@code row = index / width} and {@code column = index % width}
     * formulas.
     *
     * @param index the coordinate in a 1-dimensional array
     * @return a cell with coordinates in a 2-dimensional array
     */
    private Cell fromIndex(int index) {
        var row = index / width;
        var column = index % width;
        return new Cell(row, column, PASSAGE);
    }

    /**
     * Given the coordinates of two cells that compose an edge in
     * an imaginary edge form, it scales and transforms them to
     * the coordinates of the cell that connect passages in an
     * original form. Returns a passage cell with this coordinates.
     *
     * @param first  one edge ending
     * @param second another edge ending
     * @return a passage cell with the transformed coordinates
     */
    private Cell getPassage(Cell first, Cell second) {
        var row = first.getRow() + second.getRow() + 1;
        var column = first.getColumn() + second.getColumn() + 1;
        return new Cell(row, column, PASSAGE);
    }
}
```

6. Interfaces

The user interacts with Maze Runner through the console interface. The tool prompts the user to input the size of the maze, and the generated maze is displayed visually in the console. Maze solutions are presented, demonstrating the effectiveness of the A* algorithm in finding the shortest path. Users can save and load mazes in text format, enhancing the versatility and utility of the tool.

```
run:
=== Menu ===
1. Generate a new maze
2. Load a maze
0. Exit
1
Enter the size of the new maze (in the [size] or [height width] format)
10 20
==  ====================================
==                  ==  ==  ==      ====
==  ==========  ==  ==  ==  ==  ========
==  ==          ==  ==              ====
==========  ======  ==  ==========  ====
==  ==      ==  ==          ==      ====
==  ==  ======  ======  ======  ========
==              ==          ==      ====
==================================  ====
==================================  ====

=== Menu ===
1. Generate a new maze
2. Load a maze
3. Save the maze
4. Display the maze
5. Find the escape
0. Exit
3
Enter the filename
C:\Users\Huawei\Desktop\MAZE.txt
The maze is saved
```

```
=== Menu ===
1. Generate a new maze
2. Load a maze
3. Save the maze
4. Display the maze
5. Find the escape
0. Exit
5
==??=====================================
==??????????????????==  ==  ==      ====
==  ==========  ==??==  ==  ==  ========
==  ==          ==??==??????????????????====
==========  ======??==??==========??====
==  ==      ==  ==??????    ==??????====
==  ==  ======  ======  ======??========
==              ==          ==??????====
==================================??====
==================================??====

=== Menu ===
1. Generate a new maze
2. Load a maze
3. Save the maze
4. Display the maze
5. Find the escape
0. Exit
2
Enter the filename
C:\Users\Huawei\Desktop\MAZE.txt
The maze is loaded
```

```
=== Menu ===
1. Generate a new maze
2. Load a maze
3. Save the maze
4. Display the maze
5. Find the escape
0. Exit
5
==??=====================================
==??????????????????==  ==  ==      ====
==  ==========  ==??==  ==  ==  ========
==  ==          ==??==??????????????????====
==========  ======??==??==========??====
==  ==      ==  ==??????    ==??????====
==  ==  ======  ======  ======??========
==              ==          ==??????====
==================================??====
==================================??====

=== Menu ===
1. Generate a new maze
2. Load a maze
3. Save the maze
4. Display the maze
5. Find the escape
0. Exit
0
Goodbye!
BUILD SUCCESSFUL (total time: 27 seconds)
```

7. Conclusion

In conclusion, the "Random Maze Generator" project, Maze Runner, successfully addresses the challenge of maze generation and pathfinding. By implementing randomized Kruskal's algorithm for maze creation and the A* algorithm for solving, Maze Runner provides a versatile and efficient tool for maze enthusiasts. The project's adherence to Java and its comprehensive Javadoc contribute to its accessibility and maintainability. The console interface offers a user-friendly experience, allowing users to interact with and visualize mazes effortlessly. Maze Runner stands as an effective solution for maze generation and pathfinding, showcasing the power of algorithms in solving intricate problems.